Lecture 06: Designing a URL Shortener like Bitly

www.example.com/add /userId=38 userName=Aditya

↓

www.bit.ly/Quet23

HLD → open Ended Question





Q. How many people we are targeting ?

↳ 20 million URLs

Q. www.example.com/api/v1/userId=18 userName=Aditya

↓

→ www.bit.ly/aQw12

Q. How small the short URL?
↳ As short as possible

Q. What characters i can use to create short URL?
↳ (0-9, a-z, A-Z)

Flow:
1. User comes to our service by giving as a long URL
2. We will give him back a short URL
3. User put this short URL in browser & it's redirected to the long URL.

2 task:
Short URL generate
User Redirect

Back of the envelope calculations

QPS (Query per second)
20 million URLs generated per day.

→ write operations: 20 million/day
write operation/sec: $\frac{20 * 2^{20}}{86400 \text{ sec}}$

$= 0.000231481481 5 * 2^{20}$
$= 231.48 \times 10^{-6} \times 2^{20}$
$= 231 \times 2^{-20} \times 2^{20}$
$= \sim 231$ operation/sec

→ Read operation:
1:10 (write:read)
Read oper/sec = 231 * 10 = 2310 oper/sec

Storage Unit :

Assume for 5 years.

20 million URLs daily [records/day]

↳ 20 million * 365 days * 5 years

→ 36,500

↳ $36{,}500 * 2^{20}$ records

$36.5 * 10^3 * 2^{20}$ records

$36.5 * 2^{10} * 2^{20}$

36.5 Billion records to store

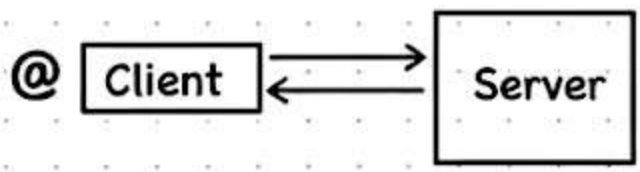
Assume 1 record = 100 Byte

→ 36.5 Billion * 100 Byte

→ $36.5 * 2^{30} * 0.1 * 2^{10}$

→ $36.5 * .65 * 2^{40}$

= 3.65 TB



API end points

1. Generate a short URL
2. Redirect to long URL

Generate a short URL :

- post : www.bit.ly/api/v1/generate

Query Param : longURL ; value

OR

Not good (api limit, itself it is long) → www.bit.ly/api/v1/generate/{{ longURL }}

OR

- post : www : bit.ly/api/v1/generate

Best → Body: {

longURL : value }

1. Generate a Short URL;

Post : www.bit.ly/api/v1/generate

Body : {

longURL : value

}

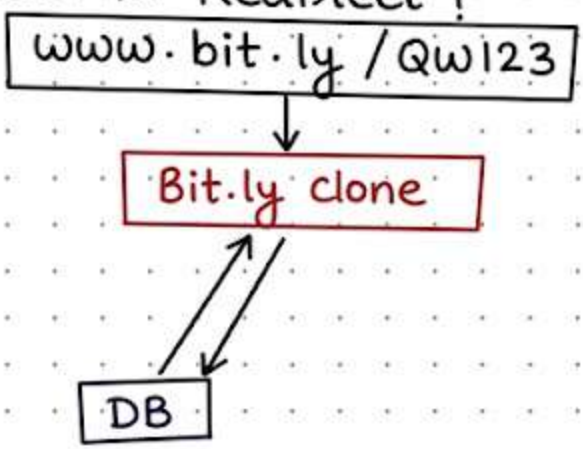
Return : Short URL

2. Redirect to LongURL

Get : www.bit.ly/{{shortURL}}

Response : Redirect to Long URL

How to Redirect ?



Status code :

301 : Redirection (Permanent redirection)

302 : Redirection (temporary redirection)

www.bit.ly/QW123 —301→ {{long URL}}

Client Side Caching

www.bit.ly/QW123 —302→ {{long URL}}

first redirect to our websike the to that long

How do we decide ? (301 or 302)

301 → if server is Small

↳ DB Read Query is Minimum

Cons : you will never know how much user use your website

302:

↳ server is scalable (large enough)

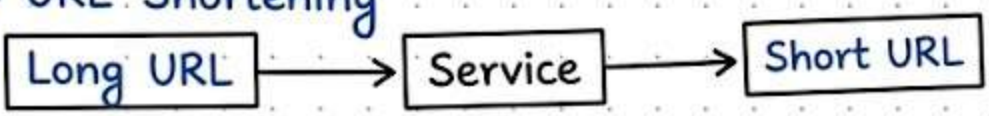
↳ DB Read Query is optimized

↳ we can have user Metrics (Records)

Design an Initial Flow

1. URL Shortening (POST)
2. URL Redirect (GET)

1. URL Shortening



post : www.bit.ly/api/v1/generate

Body { long URL }

Return : Short URL

Mapping : Long URL → Short URL

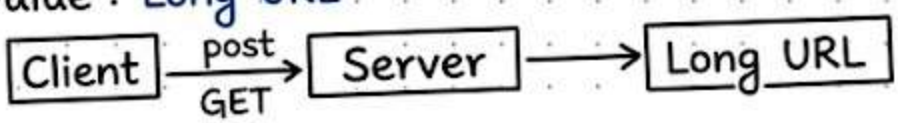
Long URL → mapped to → short URL

Hash Map / Hash table

key : Short URL

Value : Long URL

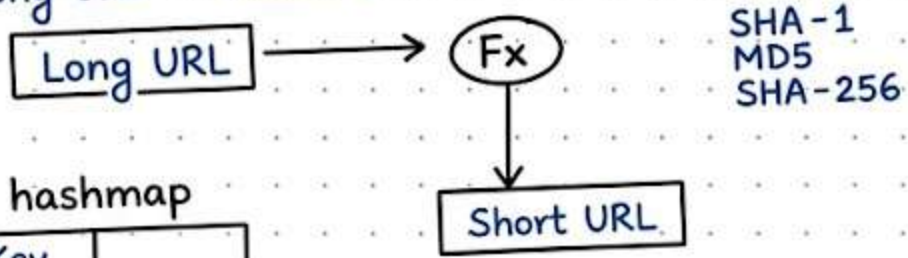
table.get (Short URL) = long URL



How to generate short URL from Long URL ??

Hash functions

Long URL → Hash function → Short URL



SHA-1
MD5
SHA-256

hashmap

Key	
Short URL	Long URL

storage stored in DB

Storing :

1. User comes with a long URL
2. We generate a short URL for the same using hash function.
3. Store the shortURL as LongURL as key and LongURL as value in our hashmap.
4. Return the short URL.

Redirecting :

1. User comes with a short URL
2. We find the equivalent Long URL by doing map.get (ShortURL)
3. Redirect the user to long URL (302 redirect)

DB : HashMap → NoSQL (Amazon Dynamo)

HLD : Distributed Applications

Multiple web servers
Multiple DB servers (Db Replication and scaling)
↓
Sharding
Consistent Hashing

Now we have 2 Hash functions :
1. Find out the shards (SHA-1)
2. Find out the shortURL from Long URL (SHA-1 or any) ?

↳ Better Design

NoSQL → SQL DB

Why ?



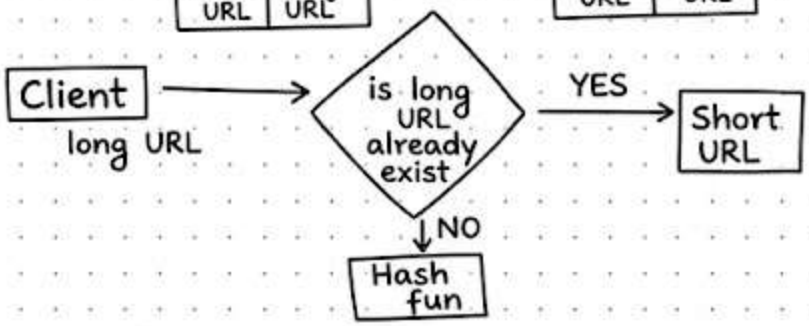
1. long URL → hash fun → Short URL

Dynamo DB

key	Value
Short URL	Long URL

Dynamo DB

key	Value
Long URL	Short URL



2. production level application → for a particular user how many URLs are generated till now.

SQL → support complex queries efficiently

3. Write operations are faster in SQL then in NOSQL.

SQL Table

ID (PK), short URL, longURL

hash functions ??

Convert a long URL to a Short URL

Hash value length :

possible symbols : (0-9, a-z, A-Z)

Total = 10 + 26 + 26 = 62 characters

Interviewer → keep it as small as possible

www.bit.ly/asd (min characters)

using permutations

www.bitly.ly/

If we take just 1 Character → 62 possible URLs
If we take just 2 Characters → 62^2
If we take just 3 → 62^3

How much should I take → n → 62^n possible URLs

36.5 Billion records to store

$62^n \geq 36.5$ Billion

n = 6 (56 Billion Records)

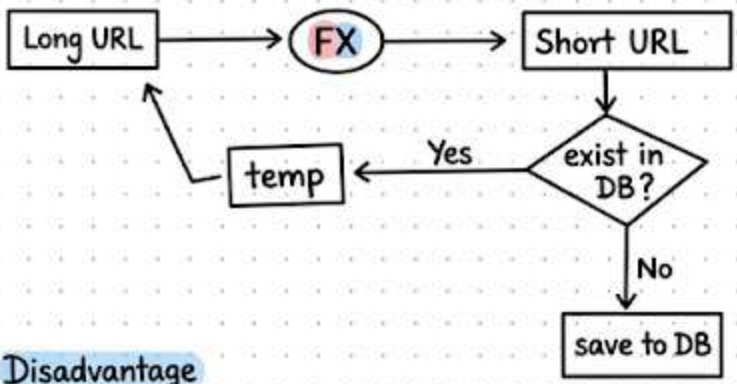
www.bitly.ly/------

N = 6 ⟶ they are giving big values

SHA-1, MD5, SHA256

1. Hashing using existing Algorithms will lead to too much collisions

Hash + Collision Removal



Disadvantage

1. DB hits increases
2. Slow (Recursive calls)

Base X Approach

Base 10 → Base 2

2 → 10

3 → 11

(62 possible Characters)

Number (Base 10) → Base 62

62 Numbers (0-9, a-z, A-Z)

0 - 0
1 - 1
9 - 9
10 - a
11 - b
35 - z
36 - A
61 - z

Convert 14320 into Base 62

62	14320	
	230	60
	3	44
	0	3

3 (44) (60)

3IY

Id	Short URL	LongURL
14320	3IY	www.example.co

Advantage :

1. Collision is impossible.

Disadvantage :

1. Need a Unique ID generator
2. URL Length is not freed
3. It is easy to figure out the next generated Short URL (if id is incremented by 1)

Example :

Long URL :

https: // www.example.com/get/code/userId = 124

↓

UniqueId : 14320

short URL : base62 (14320) = 3IY

↓

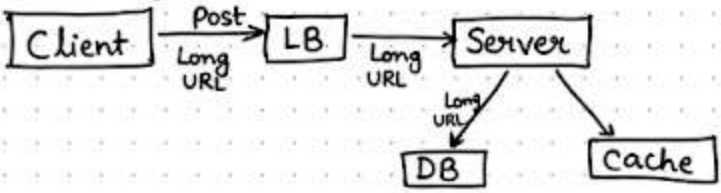
Then store it to DB

We need to build a unique ID generator

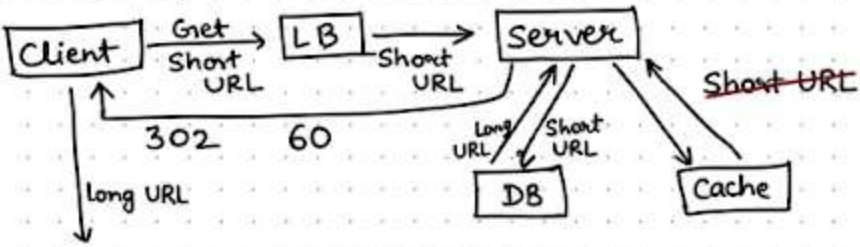
Multiple DB (shards) → ✗ Each will generate there own IDs, 2 IDs can be same

Centralized ID generator ✓

Storing :



Redirection :



What if unique ID generator generates a value that when converted to base 62 is more then 6 digit ?

~~Not possible~~

62^6 = 56 Billion unique records

36.5 Billion to be stored actually.